Numerical Methods (S210007) NM – 2026/27

I. Roko
M. Plazzogna

Assignment #4 *The answers should be submitted until Wednesday October 28, 14:00*

Exercise 1

Psimul07en

Suppose you travel to university by tram on each working day. In this stylized model, inspections on different days are independent and each day has inspection probability $p = 0.05$. An inspection is a Bernoulli "success". Use:

```
import numpy as np
rng = np.random.default_rng(0)
```

Continue the same generator sequence; do not reset the seed inside loops or functions.

1. Use `rng.uniform()` to simulate an inspection, both with and without an `if...else` statement. Store `s=1` for an inspection and `s=0` otherwise. Use the same uniform draw when comparing the two implementations.

2. Simulate 70 working days with a `for` loop. Store the daily outcomes in an array `s`, then sum its entries to obtain the number of inspections. The theoretical count is binomial with parameters 70 and p.

3. Repeat without an explicit `for` loop, using NumPy array operations. For a direct implementation check, apply both methods to the same array of uniform draws; a fresh sample need not give the same inspection count.

4. Use a `while` loop to simulate the number of days `d` until the first inspection. Generate the first draw and initialize `d=1` before the loop, so the inspected day is included. This waiting time has a geometric distribution on $1, 2, \ldots$.

5. Assume a hypothetical fine of 100 CHF and 21 working days per month. Write `daysUntilChecked(p)` to simulate and return `d` using a `while` loop, for $0 < p \leq 1$. Call it with $p = 0.05$ and compute the single-cycle rate $c = 100 \cdot 21/d$.

6. Call the function $m = 10^4$ times in a `for` loop. Store the waiting times and calculate their mean `D`. Estimate the long-run monthly fine cost as $c = 100 \cdot 21/D$. Here inspections are repeated independent cycles; this is a cost per total elapsed time, not the mean of the individual rates $2100/d$.

7. Compare this modeled cost with a hypothetical monthly pass cost of 45 CHF. Starting at $p = 0.05$, use a `while` loop to decrease p by 0.002. At each candidate probability, estimate `D` using 10^4 simulated waiting times and compute $2100/D$. Stop at the first positive grid value with estimated cost strictly below 45 CHF, and call it `p0`. Report the search results and the expected number of inspections in 70 days at `p0`. Do not simulate with $p \leq 0$; report if no admissible grid value is found.

The answer is a simulation-based grid result, not an exact largest probability over a continuous interval. As a check, use $\mathbb{E}[d] = 1/p$ to derive the theoretical monthly cost and its strict inequality with 45 CHF.

The costs above are assumptions for a mathematical exercise, not current fares or a full model of the consequences of travelling without a valid ticket.

Exercise 2

$Psimul10en$

Consider independent throws of a fair six-sided die. Use `import numpy as np`, `import matplotlib.pyplot as plt` and a random generator such as `rng = np.random.default`$_rng(0).Define$

1. `Simulate five throws with dice() and report the sum of the points, called the` *`score`*`.`
2. `Repeat the five-throw experiment` $n = 100\,000$ `times and store the scores in an array S.`
3. `Plot a histogram with bins centered on the integer scores. Use np.bincount to identify the most frequent observed score(s). Report all ties. Distinguish the empirical mode of this sample from the mode(s) of the theoretical distribution.`

`Now count the throws needed to reach a total score of at least 20.`

4. `Use a for loop with at most 20 iterations. Accumulate the points and use break as soon as the total reaches or exceeds 20. Report the number of throws.`
5. `Repeat with a while loop. Explain whether an additional iteration limit is needed under the assumption that each throw returns an integer from 1 to 6. Independent samples can give different counts.`
6. `Create scoremin(m), with one input (a positive integer target` m`) and one output (the number of throws required to reach or exceed` m`). Use one of the preceding loop procedures.`
7. `Call scoremin(20)` 1000 `times, storing the counts in an array I. Find all empirical modes of I.`
8. *`Bonus:`* `For a loaded die, let face 6 have probability` p`, and each other face probability` $(1-p)/5$`. Write diceP(p) to return one throw for` $0 \leq p \leq 1$`, using rng.choice and the six probabilities. Write scoreminP(m, p) to return the throws needed to reach or exceed target` m`.`

Starting at $p = 1/6$, increase p by 0.1 on successive iterations. At each candidate, simulate 1000 waiting counts for target 20, and report the empirical mode(s). Stop at the first candidate where 4 is among the empirical modes. Keep p in $[0, 1]$ and report if no tested value qualifies. Retain all modal ties rather than silently choosing one. This is a simulation-based grid search; its result may change with the random sample. Repeat the selected candidate and its predecessor with 10 000 experiments to assess stability.